Wroclaw University of Science and Technology

Windows Firewall, IPSec, and OpenSSH

IT Infrastructure Management (W04IST-SI0826G)

Wojciech Thomas, PhD

Department of Applied Informatics
Faculty of Information and Communication Technology

Spring 2026

This week

After this lecture, you should be able to:

- configure Windows Firewall rules and deploy them domain-wide via Group Policy
- explain how IPSec Connection Security Rules authenticate and encrypt network traffic
- configure certificate-based IPSec authentication using the Enterprise CA from the previous lecture
- install and configure OpenSSH Server on Windows Server 2025 for secure remote administration
- troubleshoot common misconfigurations in firewall rules, IPSec policy, and SSH key authentication

Why this matters

- A junior admin leaves RDP (TCP 3389) open on a production server — no firewall rule blocks it
- A password is intercepted over a cleartext management session on the same network segment
- Both incidents are preventable with the tools taught today
- **The same Windows console — `wf.msc` — is where all of it is configured**

Agenda

Windows Firewall - overview

Windows Firewall

Host-based firewall included with every Windows edition.
Enabled by default.

- Current product name: **Windows Firewall**
  - the MMC snap-in still uses “Windows Defender Firewall with Advanced Security”
- Default behaviour:
  - block all inbound
  - allow all outbound
- Stateful: no explicit return rule required for the response

Windows Firewall – profiles

- Three network profiles:
  - **Domain**
    - applied when a domain controller is reachable
    - *cannot be set manually*
  - **Private**
    - internal or home networks
    - set manually by an administrator
  - **Public** — default for unidentified networks
    - most restrictive settings
- Applied automatically per adapter
- Check active profile: `Get-NetConnectionProfile`
- Check profile settings: `Get-NetFirewallProfile -Name Domain`

Firewall rule anatomy

Property	Example values
Action	Allow / Block
Direction	Inbound / Outbound
Protocol	TCP, UDP, ICMP
Port	e.g. 443, 8080, 1024–65535
Program	e.g. `%SystemRoot%\System32\svchost.exe`
Profile	Domain, Private, Public (any combination)

```
New-NetFirewallRule -DisplayName "Allow HTTPS Inbound" `
    -Direction Inbound -Protocol TCP `
    -LocalPort 443 -Action Allow -Profile Domain

Get-NetFirewallRule -DisplayName "Allow HTTPS Inbound"
Disable-NetFirewallRule -DisplayName "Allow HTTPS Inbound"
```


Predefined vs. custom rules

- Windows ships **~500 predefined rules** grouped by Windows feature or server role
- Predefined rules activated automatically with a role, e.g.:
  - Installing *DHCP Server* opens TCP/UDP 67/68
  - Enabling *Remote Desktop* opens TCP 3389
  - Enabling *OpenSSH Server* opens TCP 22
- **Custom rules** for third-party applications and non-standard requirements
- Best practice: always check for a predefined rule before creating a custom one

```
# List all predefined rules (those belonging to a group)
Get-NetFirewallRule | Where-Object Group -ne "" |
    Select-Object DisplayName, Group, Enabled
```

Deploying firewall rules with GPO

- *Computer Configuration > Windows Settings > Security Settings > Windows Firewall with Advanced Security*
- **Local rule merge** setting controls whether local admins can add rules on top of GPO rules:
  - `AllowMerge` — local admins may add rules (default)
  - `Block` — local rules are ignored, only GPO rules apply (recommended for servers)

```
# Check merge policy per profile
Get-NetFirewallProfile | Select-Object Name,
    AllowLocalFirewallRules

# Verify which rules came from GPO vs. local config
Get-NetFirewallRule | Select-Object DisplayName,
    PolicyStoreSourceType
```


Monitoring and troubleshooting

Repeatable diagnostic workflow — follow in order:

1. Check active profile: `Get-NetConnectionProfile`
2. Check rule exists and is enabled:

```
Get-NetFirewallRule | Where-Object {
    $_.LocalPort -eq "443" -and $_.Enabled -eq "True"
}
```

3. Enable firewall logging (disabled by default):

```
Set-NetFirewallProfile -Name Domain `
    -LogAllowed True -LogBlocked True
# Log: %SystemRoot%\System32\LogFiles\Firewall\pfirewall.log
```

4. Test connectivity: `Test-NetConnection -ComputerName srv1 -Port 443`
5. Read the log — look for `DROP` entries matching the port and source IP

Agenda

What is IPSec

Internet Protocol Security (IPSec)

A suite of protocols that **authenticate** and optionally **encrypt** every IP packet exchanged between two hosts

- Windows Firewall controls *which* traffic is **permitted**
- IPSec ensures that permitted traffic is:
  - **authenticated** – you know who is the sender
  - **encrypted** – third party cannot eavesdrop on the communication
- Protects against:
  - ARP spoofing
  - man-in-the-middle interception
  - packet injection

IPSec - configuration and use cases

- Integrated into Windows Firewall
  - the same snap-in
  - deployed via the same GPO node
- Use cases:
  - DC-to-DC replication
  - admin workstation to sensitive server
  - server isolation zones

Before we continue — predict

- DC1 and CL1 are both domain-joined
- You configure a Connection Security Rule on DC1 **requiring** IPSec authentication for all traffic to and from DC1
- CL1 has **no** Connection Security Rule configured
- **Can CL1 still communicate with DC1?**
- **It depends on the rule enforcement level:**
  - *Request authentication* — DC1 tries IPSec; if CL1 cannot authenticate, traffic falls back to unprotected (communication continues)
  - *Require authentication* — DC1 refuses unprotected traffic; CL1 **cannot communicate** until it also has a matching rule
- This distinction drives the phased rollout strategy for IPSec deployment

Transport mode vs. Tunnel mode

IPSec can operate in two modes:

- Transport mode
  - Host-to-host communication
  - encrypts the **payload** only
    - the original IP header is preserved
  - Used with *Connection Security Rules*
- Tunnel mode
  - Site-to-site VPN
  - wraps the **entire IP packet** in a new outer IP packet
  - Out of scope of this lecture

IPSec authentication methods

Three options available in Connection Security Rules:

Method	How it works	When to use
Kerberos V5	Uses existing AD domain credentials	Domain-joined machines; requires reachable DC
Pre-shared key (PSK)	Both sides share a static secret string	Legacy and testing only — not recommended
Computer certificate from a CA	Machine proves identity with a certificate from a trusted CA	Recommended for production

- Certificate authentication with IKEv2 is the recommended production choice
- The Enterprise CA from Lecture 11 is exactly the trust anchor used here

Certificate requirement for IKEv2 — critical callout

When using **certificate-based authentication with IKEv2**, the machine certificate must carry a specific Extended Key Usage (EKU):

Required EKU for IKEv2 machine authentication

IP Security IKE Intermediate (OID `1.3.6.1.5.5.8.2.2`)
A standard Computer or Web Server certificate does **not** include this EKU.
IKE negotiation will **silently fail** — no error message is logged.

- Solution: duplicate the built-in **IPSec (Offline Request)** template in the CA console
- Add `IP Security IKE Intermediate` to the **Enhanced Key Usage** field of the template
- Enroll the certificate on **both** machines before configuring the Connection Security Rule

Deploying Connection Security Rules via GPO

- Connection Security Rules share the **same GPO node** as firewall rules:
  - *Computer Configuration > Windows Settings > Security Settings > Windows Firewall with Advanced Security > Connection Security Rules*
- Phased rollout strategy — never go straight to Require:
  1. Deploy with **Request authentication** — machines try IPSec
     - unprotected fallback allowed
  2. Verify all machines form SAs: `Get-NetIPsecMainModeSA` across the estate
  3. Switch to **Require authentication** — only after all machines are confirmed

Monitoring IPSec

```
# IKEv2 main mode SAs — Phase 1 (authentication)
# A result here means both machines authenticated successfully
Get-NetIPsecMainModeSA

# Quick mode SAs — Phase 2 (encryption negotiation)
# A result here means the encrypted tunnel is active
Get-NetIPsecQuickModeSA

# Security event log — SA establishment and failures
# 4650/4651: SA established   4653: SA failed
Get-WinEvent -LogName Security |
    Where-Object Id -in 4650, 4651, 4653 |
    Select-Object -First 10 TimeCreated, Id, Message
```


Common IPSec failure causes

Common failure causes (in order of frequency):

1. Certificate missing `IP Security IKE Intermediate` EKU
2. Root CA certificate not trusted on one of the two machines
3. Mismatched authentication methods between the two Connection Security Rules

Discuss with your neighbour (2 min)

- You configure a Connection Security Rule on DC1 requiring certificate authentication
- You enroll a certificate on DC1 and CL1 using the standard **Computer** template
- `Get-NetIPsecMainModeSA` returns no results after 5 minutes of traffic
- **What is the most likely cause? What is your first diagnostic step?**

- Most likely: the **Computer** template does not include the `IP Security IKE Intermediate` EKU
- First step: `certlm.msc` on DC1 → *Personal* > *Certificates* → open the machine certificate → *Details* tab → *Enhanced Key Usage* — check whether the EKU is listed
- Fix: create a custom template with the correct EKU; re-enroll both machines; verify with `Get-NetIPsecMainModeSA`

Agenda

Why SSH on Windows

Tool	Protocol	Cross-platform client	Scriptable
Remote Desktop (RDP)	RDP	Windows only	Limited
PowerShell Remoting	WS-Man	Windows-centric	Yes
OpenSSH	**SSH**	**Any OS**	**Yes**

- A Linux administrator can manage a Windows Server 2025 domain controller using `ssh`
- SSH sessions are encrypted end-to-end, auditable, low-bandwidth
- OpenSSH is a **built-in Windows feature** since WS2019
- On Windows Server 2025 it is installed by default

OpenSSH on Windows Server 2025

- On WS2025 OpenSSH is pre-installed
  - Enable with GUI: *Server Manager* > *Local Server* > *Remote SSH Access* > *Enabled*
  - Or use PowerShell:

```powershell
# Enable sshd and configure automatic startup
Get-Service -Name sshd | Set-Service -StartupType Automatic
Start-Service sshd

# The predefined firewall rule for TCP 22 is created automatically
Get-NetFirewallRule -DisplayName "OpenSSH*"
```

- Restrict SSH access to specific users or groups in `%ProgramData%\ssh\sshd_config`:

```
AllowGroups contoso\sshusers contoso\serveroperators
```

Key-based authentication

- Passwords are vulnerable to brute force and credential stuffing attacks
- A key pair uses asymmetric cryptography — the **private key never leaves the client**
- Keys survive password policy changes and can be revoked independently

```
# On the client (CL1) — generate a key pair
ssh-keygen -t ecdsa

# Two files are created:
#   private key (protect this!):
#     C:\Users\<username>\.ssh\id_ecdsa
#   public key (copy to server):
#     C:\Users\<username>\.ssh\id_ecdsa.pub
```


SSH Agent on Windows

- Use `ssh-agent` to load the private key securely:

```
Get-Service ssh-agent | Set-Service -StartupType Automatic
Start-Service ssh-agent
ssh-add $env:USERPROFILE\.ssh\id_ecdsa
```

- Agent runs in the background
- Holds decrypted keys in memory
- Provides them to `ssh` on demand

The authorized keys on Windows

Critical: different file locations depending on account type

Account type	Public key file location
Standard user	`C:\Users\<username>\.ssh\authorized_keys`
Administrators group member	**`C:\ProgramData\ssh\administrators_authorized_keys`**

- The `administrators_authorized_keys` file requires strict ACL
  - if the permissions are too broad, `sshd` **silently ignores** the file
- Set the correct ACL permissions:

```
icacls.exe `
    "C:\ProgramData\ssh\administrators_authorized_keys" `
    /inheritance:r `
    /grant "Administrators:F" `
    /grant "SYSTEM:F"
```


Setting PowerShell as the default shell

Default shell after SSH login is `cmd.exe`. Change it via registry:

```
# Set Windows PowerShell 5.1 as the default SSH shell
# powershell.exe is always present on WS2025
New-ItemProperty -Path "HKLM:\SOFTWARE\OpenSSH" `
    -Name DefaultShell `
    -Value "C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe" `
    -PropertyType String -Force

Restart-Service sshd
```

To use PowerShell 7 (`pwsh.exe`) instead — install first, then set the path:

```
winget install Microsoft.PowerShell
# Then set Value to: C:\Program Files\PowerShell\7\pwsh.exe
```

Agenda

1. Windows Firewall
2. IPSec
3. OpenSSH
4. Closing

In the lab

In the lab you will configure all three layers of secure communication:

1. **Firewall** — create a custom inbound rule; deploy via GPO to CL1; verify with `Test-NetConnection` and the firewall log
2. **IPSec** — create a certificate template with the `IP Security IKE Intermediate` EKU; enroll on DC1 and CL1; configure a Connection Security Rule; verify with `Get-NetIPsecMainModeSA`
3. **OpenSSH** — enable `sshd` on DC1; deploy the admin public key to `administrators_authorized_keys` with correct ACL; set PowerShell as the default shell; verify passwordless login from CL1

Bibliography

[1] O. Thomas, *Windows server inside out: Updated for windows server 2025*. Microsoft Press, 2025.

[2] J. Krause, *Mastering windows server 2025 - fifth edition*. Packt Publishing, 2025.

[3] "Windows firewall overview," 2026. https://learn.microsoft.com/en-us/windows/security/operating-system-security/network-security/windows-firewall/ (accessed May 31, 2026).

[4] "OpenSSH for windows documentation," 2025. https://learn.microsoft.com/en-us/windows-server/administration/openssh/openssh-overview (accessed May 31, 2026).